A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing

QI YU, School of Cyber Science and Engineering, Southeast University, PR China

WEI LI*, School of Computer Science and Engineering, Southeast University, PR China

Integrated scheduling of processing and transport in flexible job shops almost always treats travel times as a location-indexed constant matrix. Once a finite fleet shares a sparse corridor network, travel time is the outcome of conflict-free path planning—routing, in this paper—because one job's occupancy delays others. This paper studies a class in which each operation may use one of several robotic arms that are homogeneous in capability and heterogeneous in efficiency, while AGVs must travel conflict-free. Machine flexibility then becomes an operational trade-off: when the corridor to a fast arm is congested, a slower arm with better connectivity may be the better assignment.

A *closed-loop bilevel framework* connects the two decisions along an evaluation path and a decision path. The upper level assigns machines and sequences operations; the lower level generates conflict-free paths by time-window reservation. *Routing-in-the-loop evaluation* takes the fitness of each candidate as the makespan after that routing, so yielding wait enters the search objective. *Reservation-table-aware dispatching* selects a vehicle by trial-querying both transport legs on the reservation table, not by unimpeded travel time. Two low-overhead optimizations—admissible lower-bound pruning and winner-path reuse—change neither the selected vehicle nor the timetable.

On 10 controlled instances of one size, with contention from 6.6% to 33.5%, a four-level ladder, 10 seeds, and an equal wall-clock budget, the closed loop reduces makespan by 18.86% ($p < 10^{-4}$) relative to the open loop; within the closed loop, reservation-table-aware dispatching reduces it by a further 2.45% ($p = 0.0054$); end-to-end the reduction is 21.35% (94 wins, 5 losses, $p < 10^{-4}$), close to the independent product 22.6%. The closed loop is the main effect and remains significant per instance on 9 instances after correction; dispatching is a much smaller secondary effect, significant only when all instances are pooled and on none individually. The two are near-additive; no positive interaction is detected.

Additional Key Words and Phrases: flexible job shop scheduling, conflict-free routing, closed-loop bilevel framework, routing-in-the-loop evaluation, reservation-table-aware vehicle dispatching

ACM Reference Format:

Qi Yu and Wei Li. 2026. A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing. 1, 1 (September 2026), 26 pages. https://doi.org/XXXXXXX.XXXXXXX

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

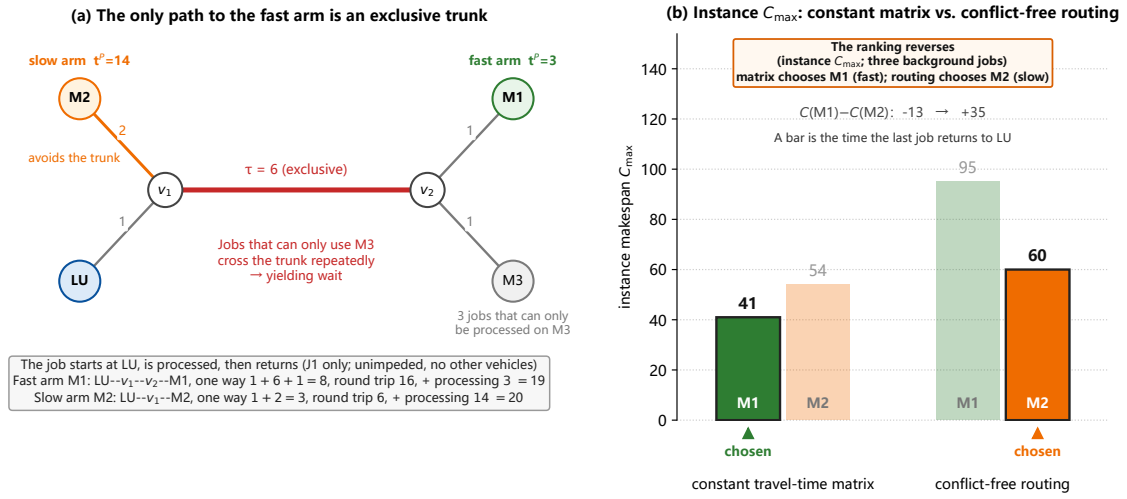


Fig. 1. Motivating instance: two readings of the same pair of assignments. (a) J1's unimpeded chain: exclusive trunk to M1 ($2(1+6+1)+3=19$) versus the short corridor to M2 ($2(1+2)+14=20$); the bottom box is this comparison, not the bars in (b). (b) Instance $C_{\max}$ (last return to LU): the constant matrix ranks M1 better (41 vs 54); conflict-free routing ranks M2 better (95 vs 60). An *existence demonstration*: of 1200 nearby parameter combinations, 205 (17%) reverse.

1 Introduction

1.1 Motivation and gap

Flexible job shop scheduling with transportation has long treated travel time as a constant, read from a matrix indexed by location pairs—the dominant convention [22], still widely used [4, 14, 16]. The assumption is reasonable when vehicles rarely meet. Real shops are not built that way: a few vehicles share a sparse corridor network, and a single-lane segment cannot be used in both directions at once; once fleet density is high enough, the fleet's own traffic becomes the main delay [9]. Travel time is then the outcome of two decisions—which path a vehicle takes (conflict-free path planning; routing, in this paper) and who first reserved the contended segment—not the distance between two points.

That coupling can be optimized only if the scheduler can change the assignment. When an operation may use any arm in an option set, and the processing time depends on the arm, the assignment selects both the destination and the duration. Under a constant matrix the comparison is arithmetic (shorter processing plus longer travel, or the reverse). With corridor contention the wait on the path to a fast arm is invisible to the matrix, so a slower arm with better connectivity may be better—and the ranking may flip again at a different congestion level. The trade-off needs both ingredients: heterogeneous optional arms, and distinct paths on which vehicles contend.

Figure 1 reports two readings on a recomputable instance (4 jobs, 2 AGVs). The first is the unimpeded chain of J1 alone (panel (a)): LU $\to v_1 \to v_2 \to$ M1 is $2(1+\tau+1)+3=19$; LU $\to v_1 \to$ M2 is $2(1+2)+14=20$. The fast arm wins that comparison iff $19<20$, so $\tau=6$ and $t^P_{\text{fast}}=3$ already force $t^P_{\text{slow}}\geq 14$. The second is instance $C_{\max}$ (panel (b)): last return to LU, same decoder, not 19 or 20 and not a sum of processing times. On the constant matrix the two AGVs stay in step if J1 goes to M1 (last return 41) and one AGV is held at M2 if J1 goes there (last return 54). Conflict-free routing on the same pair occupies the exclusive trunk when J1 uses M1 (last return 95) and avoids it when J1 uses M2 (last return 60). Neither assignment nor sequence has changed; only the evaluation has.

Existing work typically treats flexible-job-shop scheduling or conflict-free travel, not both; when they are combined, the assignment is usually fixed first and routing then absorbs the contention, so delay cannot revise the schedule. An exact method already formulates both in one problem and solves small instances to proven optimality [21]; this paper adopts that problem setting, and its contribution is the solver, not the problem definition. Beyond that exact size, heuristics still schedule then resolve [19], or write realized durations back onto the time axis of the current solution only, on a benchmark with one machine per operation [10]. The trade-off in which flexibility, heterogeneous efficiency, and corridor contention are present at once, and every candidate is ranked by the makespan after yielding wait, is therefore still open at heuristic size. The literature position is in Section 2.

1.2 Approach and contributions

This paper connects the two levels along an evaluation path and a decision path, and measures each under an equal wall-clock budget. The upper level assigns machines and sequences operations; the lower level generates conflict-free paths by time-window reservation.

The *evaluation path* takes the fitness of each candidate as the makespan after conflict-free routing, so yielding wait enters the search objective. One such decode is more than an order of magnitude more expensive than a lookup of the unimpeded matrix; under a fixed compute budget, a higher cost per evaluation leaves fewer evaluations. [10] lists a fully iterative loop as future work, but does not report that cost or its effect on makespan.

The *decision path* uses *reservation-table-aware dispatching*: both transport legs (empty pickup, then loaded delivery) are trial-queried on the reservation table without writing the timetable, and the vehicle with the earliest feasible delivery is selected, replacing the idle-first rule of [10]. Two output-preserving optimizations—admissible lower-bound pruning and winner-path reuse—keep each evaluation low-overhead. A surcharge on occupied corridor–time slots, and reassignment or staggering from the yield record, were also implemented; neither further reduces makespan under the same budget (Section 5).

Four configurations in a 2×2 design (open or closed loop $\times$ rule-based or reservation-table-aware dispatching) separate the two paths. **B0** is open-loop planning plus actual execution [10]; **B0+** replaces only vehicle assignment on a frozen schedule [19]; **B1** is closed-loop planning plus rule-based dispatching; **B2** is the method of this paper. Every schedule is executed by the same conflict-free planner; what is compared is the realized makespan. Moving congestion into the search reduces makespan by 18.86% ($p < 10^{-4}$), the larger effect; within the closed loop, reservation-table-aware dispatching reduces it by a further 2.45% ($p = 0.0054$), a smaller but still significant effect; end-to-end the reduction is 21.35% (94 wins, 5 losses, $p < 10^{-4}$). The two speed-ups give a factor of 2.33, cut router calls by 47.4%, and lower the cost multiple from 12.2 to 5.23. The two factors are near-additive (21.35% close to 22.6%) and are not substitutes: open-loop dispatching gives only 4.59% ($p < 10^{-4}$). After Holm correction by family, the closed loop remains significant on 9/10 instances; dispatching is significant only when instances are pooled and on none individually (2 before correction). With 10 seeds per instance, the sample cannot yet determine on which class of instances the dispatching gain is stable. The protocol is in Section 5.1.2; per-instance results are in Section 5.4.

The four contributions, restated with supporting numbers in Section 6.1, are: **(1) Instances:** a controllable characterization of the reassignment trade-off—how high contention must be before counting yielding wait in the schedule yields a gain; **(2) Evaluation:** routing-in-the-loop evaluation, so every candidate is ranked by the makespan after conflict-free routing; **(3) Dispatching:** reservation-table-aware dispatching with two output-preserving speed-ups, so the trial query remains affordable under the same wall-clock budget; **(4) Evidence:** the two paths differ in magnitude, are near-additive,

and are not substitutes—the closed loop is larger and robust per instance, while dispatching is significant only when pooled.

The notation of the paper is collected in Table 2.

2 Related work

Two strands sit next to this paper: flexible job shop scheduling with transportation times, and conflict-free routing of exogenous tasks. Work that writes both into one problem is scarce; Table 1 places the present solver against that literature.

2.1 Two strands

FJSP_PT extends FJSP by bringing material movement into the schedule [1, 22]. Public benchmarks are collected by Homayouni and Fontes [7]. The dominant convention is a constant travel-time matrix: a vehicle needs a fixed $t(a, b)$, independent of corridor occupancy, so vehicles enter only as capacity resources. Meng et al. [14] close 29 previously open benchmarks and improve 27 best-known solutions; Qin et al. [16] guide a local search by AGVs and a critical path still computed on matrix times; Fu et al. [4] decide production and logistics jointly, without making transport duration depend on the schedule. These assumptions "reduce the complexity of the problem and even change its nature" [22]: when the fleet is large enough, arrival times are affected by congestion, and a plan that ignores that dependence need not be feasible on the actual corridors. This strand has machine flexibility, but no yielding wait.

The other strand takes corridor contention as the object. Optimal multi-agent path finding is NP-hard [5]; Safe Interval Path Planning extends single-vehicle routing to continuous-time road networks [8]. Zhang et al. [25] steer lifelong agents with offline-learned edge weights; Kim and Kim [9] give density-based congestion-aware dispatching. Surveys list dispatching as independent of path planning [12] and task allocation as adjacent to it [3]. Release times and endpoints are inputs: the router cannot change the assignment that produced the tasks. Congestion is usually a proxy—local density [9] or offline edge weights [25]—not the yield that names a corridor, an interval, and the transport that occupied it first. This strand has genuine conflict-free routing, but no flexibility.

2.2 Integration and our position

van Os and Basten [21] formalize FJSP with conflict-free transportation: optional machines, location-dependent processing times, and conflict-free travel. A logic-based Benders solver (constraint-programming master, integer-programming routing subproblem, conflict-driven cuts) solves most benchmarks to proven optimality and improves about half of them by at least 10% over prior heuristics, within the size the exact decomposition can reach. This paper adopts that setting; the contribution is a heuristic that, beyond that size, sends routing delay back into every scheduling candidate.

Sun et al. [19] still fix assignment and sequence first, then resolve conflicts with a genetic AGV encoding and time-window Dijkstra; the second stage cannot revise the first. Li et al. [10] write realized durations back onto the current solution only, on a one-machine-per-operation benchmark. The remaining gap is how the routing result enters the schedule: whether every candidate is evaluated by actual routing, and whether the assignment can change from that evaluation.

Critical-chain local search is mature [15, 16, 20], but a chain on a constant matrix cannot name a corridor; after time-window reservation it can. On instance design, [22] notes that published instances "differ mainly in size" and lists $\bar{T}_t/\bar{T}_p$ as likely to affect performance; this paper also varies how much of the congestion is assignment-related (Section 5).

Table 1. Related work versus this paper. Machine heterogeneity: optional machines with machine-dependent processing times. Coupling frequency: how often, and in which direction, the routing result enters the schedule.

Work class	Ref.	Machine het.	Vehicle contention	Coupling freq.	Feedback content
FJSP_PT metaheuristics	[4, 14, 16]	yes	constant matrix	—	—
Lifelong MAPF	[5, 25]	—	conflict-free	tasks exogenous	edge weights
Congestion-aware dispatching	[9]	no	density proxy	one-way	congestion penalty
Two-stage FJSP + routing	[19]	yes	conflict-free	one-way	none
RL scheduling + PBS	[10]	no	conflict-free	one-shot correction	arrival times
Exact LBBD	[21]	yes	conflict-free	master–subproblem	conflict-driven cuts
This paper		yes	conflict-free	every candidate	realized time + trial query

Table 2. Notation.

Symbol	Meaning
$J = \{1, \ldots, n\}$, O_{ji}	jobs; i-th operation of job j, $i \le n_j$
$M = \{1, \ldots, N_M\}$, $\Omega_{ji} \subseteq M$	robotic arms; option set of O_{ji}
t^P_{jim}	processing time of O_{ji} on arm $m \in \Omega_{ji}$
$K = \{1, \ldots, N_A\}$	homogeneous unit-load vehicles
$G = (V, E)$, τ_e	shop-floor graph; traversal time of corridor e
v_0, $\mathrm{loc}(m)$	load/unload point; node of arm m
$t^*(a, b)$	shortest-path time $a \to b$ in G, ignoring contention
$\delta_{\mathrm{ret}} \in \{0, 1\}$	finished job returns to v_0 (baseline 1)
$S_{ji}, C_{ji}, A_{ji}, C_{\max}$	start, completion, arrival; makespan

3 Problem statement and model

3.1 Problem statement

An instance is a set of jobs, robotic arms, and vehicles on a shop-floor graph (Table 2). Each job is a fixed operation chain; each operation has an option set of arms with machine-dependent processing times. Arms occupy fixed nodes; the fleet moves jobs between them. Four decisions are taken at once: (i) assign each operation to an optional arm; (ii) sequence each arm; (iii) assign and sequence the induced transport tasks on vehicles; (iv) plan a conflict-free path for every move, waits allowed. The objective is to minimize $C_{\max}$.

3.2 Modelling assumptions

A1. Static instance and complete information. All n jobs are at v_0 at $t = 0$; no release or due dates. Processing data, G, and vehicle states are known.

A2. Jobs. Fixed precedence chain per job. Processing and transport are non-preemptive; jobs cannot be split; no exogenous precedence among jobs.

A3. Robotic arms: partial flexibility and heterogeneous times. O_{ji} may run on any $m \in \Omega_{ji}$ (most operations have $|\Omega_{ji}| \ge 2$; $\Omega_{ji} = M$ is a special case). Times t^P_{jim} are machine-dependent and need not be proportional across arms. An arm processes one operation at a time, does not fail, and sequence-dependent setup is folded into t^P_{jim}.

A4. Buffers and loading. Ample input/output buffers and v_0 storage, so no blocking. Loading and unloading are negligible and do not occupy the arm.

A5. Vehicles. N_A homogeneous unit-load vehicles at constant speed, empty or loaded; no acceleration or battery. All stand empty at v_0 at $t = 0$. A task is a one-way move—v_0 to the first arm, between successive distinct arms, or, when $\delta_{\mathrm{ret}} = 1$, last arm back to v_0; any vehicle may take any task. After unloading, a vehicle is not forced back to v_0; routing may postpone its departure.

A6. Road network and conflict-freedom. G is strongly connected; each τ_e is a known constant. Corridors are *exclusive*: both directions of e share one reservation, so at most one vehicle occupies e at a time (no head-on, no overtaking). Occupancy is $[t,\ t+\tau_e)$; a second vehicle may enter at $t+\tau_e$. Nodes have ample berths, separate from the traversal resource; waits occur at berths and are always permitted.

A7. Out of scope. Machine and vehicle failures, finite buffers and blocking, battery charging, and dynamic arrivals [6, 23] are left to future work (item (5) of Section 6.2).

A1, A2, A4, and A5 are standard in FJSP_PT [22]. What shapes this paper is A3 (machine-dependent times on partially flexible arms) and A6 (contention as an exclusive-resource constraint, not a constant matrix): transport duration is an output of routing, not a fixed input. Deadlock-freedom is not assumed. A4 and A6, with Proposition 2 in Section 4.3, imply that any assignment and sequence decodes to a conflict-free, deadlock-free timetable, so the search need not repair infeasible solutions.

3.3 Decision variables and induced transport tasks

Let $x_{jim} \in \{0,1\}$ indicate that O_{ji} is assigned to robotic arm m; write $\mathrm{m}(j,i)$ for the chosen arm and $d_{ji} = \mathrm{loc}(\mathrm{m}(j,i))$ for the node of that arm. When $\delta_{\mathrm{ret}} = 1$, a zero-duration dummy operation O_{j,n_j+1} at v_0 is appended so that the return move uses the same task notation, and $C_{\max}$ is always the completion of each job's last operation.

Transport tasks are not an input; they are *induced* by the assignment:

$$\mathcal{T}(x) = \Big\{ (j,i) \ :\ d_{j,i-1} \neq d_{ji} \Big\}, \qquad d_{j0} \equiv v_0, \tag{1}$$

where task (j,i) moves job j from $d_{j,i-1}$ to d_{ji} and becomes ready at time $r_{ji} = C_{j,i-1}$ (with the convention $C_{j0} = 0$). If two successive operations are placed on the same arm, no transport task is created. Thus both the *number* and the *endpoints* of transport tasks are consequences of decision group (i): an assignment that keeps a job on the same arm throughout needs only two moves, whereas an assignment that changes arm at every operation needs $n_j + 1$ moves. Assignment of tasks to vehicles is indicated by $y_{tk} \in \{0,1\}$, with $\sum_{k\in K} y_{tk} = 1$ for each $t \in \mathcal{T}(x)$; tasks given to the same vehicle are ordered into a sequence.

3.4 Timing relations and feasibility

Write $p_{ji} = \sum_{m\in\Omega_{ji}} x_{jim} t^{P}_{jim}$ for the processing time on the chosen arm. Assignment, precedence, and the unit-capacity of each arm give

$$\sum_{m\in\Omega_{ji}} x_{jim} = 1, \qquad \forall j,i, \tag{2}$$

$$C_{ji} = S_{ji} + p_{ji}, \qquad \forall j,i, \tag{3}$$

$$S_{ji} \ \geq\ A_{ji}, \qquad \forall j,i, \tag{4}$$

$$S_{j'i'} \ \geq\ C_{ji} \ \ \vee \ \ S_{ji} \ \geq\ C_{j'i'} \qquad \forall (j,i) \neq (j',i') \text{ on the same arm}, \tag{5}$$

where A_{ji} is the time at which the job actually appears at d_{ji}. If $(j,i) \notin \mathcal{T}(x)$, the job is already there and $A_{ji} = C_{j,i-1}$. Otherwise A_{ji} is produced by the vehicle that carries task (j,i): write a^{E}_{ji} for the time at which that vehicle reaches the pickup node after its empty leg, and set

$$\ell_{ji} = \max\big(a^{\mathrm{E}}_{ji},\ r_{ji}\big), \qquad A_{ji} = \text{arrival time of the loaded leg departing at } \ell_{ji}, \tag{6}$$

i.e. the vehicle may wait for the job, and the job may wait for the vehicle. Whether empty or loaded, each leg is a walk $(e_1, \ldots, e_q)$ in G whose corridor entry times $\eta_1, \ldots, \eta_q$ satisfy

$$\eta_1 \;\geq\; \sigma, \qquad \eta_{l+1} \;\geq\; \eta_l + \tau_{e_l}, \quad l = 1, \ldots, q-1, \tag{7}$$

where σ is the earliest start of the leg; a gap $\eta_{l+1} - \eta_l - \tau_{e_l}$ is a wait at a berth. The leg ends at $\eta_q + \tau_{e_q}$. Any two distinct occupancies of the same corridor e—regardless of direction or vehicle—must be disjoint:

$$\big[\eta,\ \eta + \tau_e\big) \;\cap\; \big[\eta',\ \eta' + \tau_e\big) \;=\; \emptyset. \tag{8}$$

The objective is

$$\min\ C_{\max}, \qquad C_{\max} = \max_{j \in J}\ C_{j,\, n_j + \delta_{\text{ret}}}. \tag{9}$$

Two facts are used in Section 4. In any semi-active schedule, (4) and (5) tighten to

$$S_{ji} = \max\big(A_{ji},\ F_{\mathrm{m}(j,i)}\big), \tag{10}$$

where F_m is the completion of the preceding operation on arm m; later sections attribute a postponed start to material or to the arm by which argument is larger. And (8) is the only constraint that couples jobs through *space* rather than a shared machine.

3.5 Relation to FJSP_PT

After (8) is removed, each leg may take a shortest path, durations collapse to $t^*(a, b)$, and the model becomes FJSP_PT—the two problems differ only in corridor exclusivity; t^* is later a lower bound and the open-loop arrival estimate. van Os and Basten [21] reserve *nodes*; this paper reserves *edges*. Lyu et al. [13] use edge exclusivity and also require that a node hold at most one vehicle; this paper does not, and waits occur at ample berths. Why the instances cannot be compared directly is limitation (1) of Section 6.2.

3.6 Problem hardness

Proposition 1. *The problem defined by (2)–(9) is strongly NP-hard; it remains so when every operation satisfies* $\Omega_{ji} = M$.

Proof. Restrict to $\tau_e = 0$ for every e, each arm on a dedicated corridor from v_0, and $N_A = n$. Every leg then has duration zero, (8) holds identically, and $A_{ji} = C_{j,i-1}$. What remains is (2)–(5) with $A_{ji} = C_{j,i-1}$, i.e. FJSP with unrelated machines. Two special cases are already strongly NP-hard: $|\Omega_{ji}| = 1$ is the classical job shop (already with three machines); $\Omega_{ji} = M$ and $n_j = 1$ is makespan on unrelated parallel machines, strongly NP-hard when the number of machines is part of the input. FJSP itself is strongly NP-hard [1]. Each restriction is a polynomial reduction, so the general problem is strongly NP-hard; full flexibility does not make it easy. □

The reduction of Proposition 1 excludes vehicle contention, so the problem is already strongly NP-hard without it. Fixing decisions (i)–(iii) still leaves a conflict-free multi-agent routing problem [5]; routing adds constraints on top of the scheduling hardness. The running time of one decode is reported with the framework (Section 4.7), not here.

3.7 Instance features

Instance size is held fixed and only structure is varied; the effect of size is limitation (1) of Section 6.2. Four quantities are standard in the scheduling literature; Xin et al. [22] note that $\bar{T}_t/\bar{T}_p$ in particular is likely to affect performance.

$\bar{T}_t/\bar{T}_p$ is mean shortest-path travel among arm nodes over mean processing time. Heterogeneity H is the coefficient of variation of each operation's processing times over its optional arms, then averaged (singleton option sets contribute 0); $H = 0$ means reassignment changes no processing time, only the corridor. Flexibility $F = \text{mean}_{ji}\,|\Omega_{ji}|/N_M$ is the scope of reassignment. N_A/N_M is the vehicle-to-arm ratio (smaller: transport bottleneck more likely).

Two further quantities exist only on an explicit road network; what matters is how much congestion reassignment can avoid. Contention intensity is the share of travel time that is yielding wait on the initial random population—read off the structure, not supplied as input—and is used to order instances (Table 4). Let $\text{cut}(v_0)$ be a smallest set of corridors whose deletion disconnects v_0 from every arm; $|\text{cut}(v_0)|$ (funnel width) bounds how many vehicles can leave v_0 at once, and the layout factor varies it by adding or removing parallel exits (`funnel` halves the `high` value). A funnel share φ and a far-group cut were also examined but did not separate the intended layout classes, so they are not used (item (1) of Section 6.2).

Each instance carries a composite lower bound $\text{LB} = \max(\text{LB}_{\text{chain}}, \text{LB}_{\text{load}}, \text{LB}_{\text{cut}})$: the job-chain relaxation ignores inter-arm moves and queues; the load relaxation ignores precedence and transport; the cut relaxation spreads $X = 2n$ crossings of $\text{cut}(v_0)$ (when $\delta_{\text{ret}} = 1$) so as to minimize the load on the busiest corridor,

$$C_{\max} \;\geq\; X \Big/ \sum_{e \in \text{cut}(v_0)} \tau_e^{-1}. \tag{11}$$

On these instances LB_{cut} is largest on eight of ten and LB_{chain} on the other two. The bound is fairly loose (gap 27% to 58% to the best known solution) and is used only to check the order of magnitude, not to certify near-optimality.

4 Closed-loop bilevel framework

4.1 Overview: routing-in-the-loop evaluation and the decision path

The framework assigns the four decision groups of Section 3 to two levels (Figure 2). The upper level searches assignment and sequencing with a memetic algorithm [24]; an event-driven decoder turns a candidate into precedence-constrained transport tasks; the lower level plans conflict-free paths by time-window reservation. The output is an executable timetable and its makespan. The evaluation path (blue) returns that makespan as fitness, once per candidate, yielding wait included. The decision path (orange) chooses a vehicle by a two-leg trial query on the reservation table—empty pickup, then loaded delivery; the query does not write the timetable—rather than by an unimpeded-time estimate. Two further interfaces—a surcharge on occupied corridor–time slots, and reassignment from the yield record—serve as controls (Sections 4.4.1 and 4.6); under an equal wall-clock budget neither further reduces makespan.

4.2 Lower level: time-window reservation and conflict-free routing

The only contended resource is the corridor (A6). The reservation table maps each corridor to a time-ordered list of half-open occupancies $[t,\ t + \tau_e)$, labelled by vehicle and task, and supports four operations: earliest feasible entry no earlier than a given instant; write; revoke a task; and checkpoint/rollback for a trial query. Nodes are not reserved: traversal takes zero time, waits occur at berths (A6). The router and an independent checker inspect only corridor occupancy.

A leg is planned by a label-setting search (Algorithm 1). A label (u, t) means standing at the berth of u at time t; the cost is t. Querying the table for the earliest entry $t' \geq t$ produces the wait $t' - t$ of (7) directly. Within one decode, written paths are not revoked, so vehicles are planned in decoder order and a later vehicle yields to an earlier one.

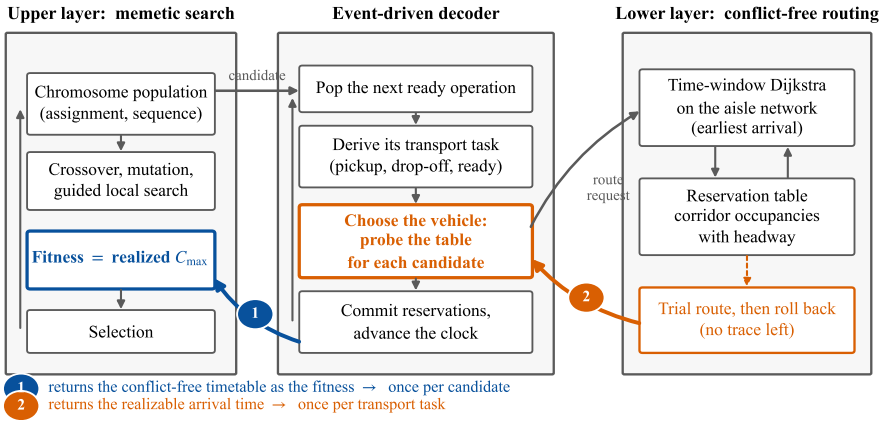


Fig. 2. The closed-loop bilevel framework. Blue path 1 returns the conflict-free timetable as fitness, once per candidate. Orange path 2 runs a two-leg trial query for the earliest feasible delivery before a vehicle is chosen; the trial query does not write the final timetable. The ladder of Section 5 is arranged by whether these two paths are open.

Algorithm 1 ROUTE: conflict-free planning of a single leg

Require: origin s, destination g, earliest departure t_0, table R, commit flag
Ensure: occupancy intervals; written to R if commit
push (s, t_0) onto a priority queue keyed by (time, node id)
while the queue is nonempty **do**
  extract the least-key label (u, t)
  if $u = g$ **then**
    backtrack occupancies; if commit, write them to R; **return** t
  end if
  for all corridors $e = (u, v)$ leaving u **do**
    $t_{\text{in}} \leftarrow R.\text{EARLIESTENTRY}(e, t, \tau_e)$
    if $t_{\text{in}} + \tau_e$ is strictly earlier than the incumbent at v, update $(v,\ t_{\text{in}} + \tau_e)$
  end for
end while
return failure ▷ G strongly connected: see text

LEMMA 1 (EARLIEST ARRIVAL IS SUFFICIENT). *Keeping only the earliest-arrival label at each node is sufficient to obtain a path that reaches the destination earliest under the current reservation table.*

PROOF. Berths have ample capacity and do not occupy the traversal resource (A6), so a vehicle that has stood at u since time t need only wait further until t' in order to take any continuation that a vehicle arriving at u only at $t' > t$ could take. An earlier-arrival label is therefore never worse, and each node need keep only the earliest arrival. Moreover, the later the query time to the reservation table, the fewer the still-feasible entry slots, so the time to enter corridor e and finish traversing it, $t \mapsto R.\text{EARLIESTENTRY}(e, t, \tau_e) + \tau_e$, is non-decreasing in t. The search extracts labels in increasing order of arrival time; the first time the destination is extracted is the earliest arrival. □

Ordinary Dijkstra therefore suffices; there is no need for the per-interval labelling of Safe Interval Path Planning (SIPP) [8]. A second criterion (a surcharge on busy slots) would force a multi-label Pareto search; Section 4.4.1 returns to that cost.

A path always exists: reservations are finite, time is bounded, and G is strongly connected, so in the worst case the vehicle waits until every occupancy has ended and then takes an unimpeded shortest path. That is the existence half of

Algorithm 2 DECODE: fitness from a complete conflict-free simulation

Require: instance, assignment x, operation sequence
Ensure: makespan, timetable, per-corridor yield statistics
$F_m \leftarrow 0$ for every arm; $\text{pos}_j \leftarrow v_0$, $\text{ready}_j \leftarrow 0$ for every job
$\text{loc}_k \leftarrow v_0$, $\text{avail}_k \leftarrow 0$ for every vehicle; clear R
for all operations O_{ji} in sequence order (dummies included) **do**
  $m \leftarrow$ arm of O_{ji} under x; $d \leftarrow \text{loc}(m)$
  if $\text{pos}_j = d$ **then**
    $A_{ji} \leftarrow \text{ready}_j$
  else
    choose vehicle k (Section 4.4)
    $a^{\text{E}} \leftarrow$ ROUTE$(\text{loc}_k, \text{pos}_j, \text{avail}_k, R, \text{true})$
    $\ell \leftarrow \max(a^{\text{E}}, \text{ready}_j)$
    $A_{ji} \leftarrow$ ROUTE$(\text{pos}_j, d, \ell, R, \text{true})$; $\text{loc}_k \leftarrow d$; $\text{avail}_k \leftarrow A_{ji}$
    accumulate yielding wait by corridor
  end if
  `bind` $\leftarrow$ `machine` if $F_m > A_{ji}$, else `arrive`
  $S_{ji} \leftarrow \max(A_{ji}, F_m)$; $C_{ji} \leftarrow S_{ji} + p_{ji}$; $F_m \leftarrow C_{ji}$; $\text{pos}_j \leftarrow d$; $\text{ready}_j \leftarrow C_{ji}$
end for
return $C_{\max} = \max_{j,i} C_{ji}$, the timetable, and the yield statistics

Proposition 2. A single call costs $O\big(|E|\,(W^2 + \log|V|)\big)$: at most $|E|$ corridors are examined; for each, $O(W)$ candidate entries are tested for overlap in $O(W)$ time, plus a heap step of $O(\log|V|)$, where W is the mean number of reserved intervals per corridor. A logarithmic earliest-entry query would drop this to $O\big(|E|(\log W + \log|V|)\big)$; this paper uses the linear scan.

The same search on an empty table over all pickup–delivery pairs yields the unimpeded matrix t^* of Section 3, used for open-loop arrival estimates, the reassignment neighbourhood, the two-stage baseline, and the admissible bound of Section 4.5.

4.3 Upper level: encoding and event-driven decoding

Let N be the number of operations. An individual has two segments: a machine-assignment vector of length N, the k-th entry an arm from Ω_{ji}; and an operation sequence in which job j appears n_j times, the x-th appearance standing for O_{jx}. The relative order of operations that share an arm is read from the sequence. When $\delta_{\text{ret}} = 1$, each job appears once more for the dummy of Section 3, so return transport needs no extra decoder branch. Vehicle assignment is decided at decode time (Section 4.4): a third segment would enlarge the search space and would no longer guarantee that every chromosome is decodable.

The decoder (Algorithm 2) *is* the evaluation path. It sweeps the operation sequence from left to right, maintaining the idle time of each robotic arm, the current node and ready time of each job, and the current node and available time of each vehicle. When the arm of an operation differs from the job's current location, a transport task of (1) is generated, a vehicle is chosen, and two paths are planned against the *live* reservation table: an empty leg to the pickup and a loaded leg to the destination, joined by (6). The operation then starts by (10); the decoder also records which of the two arguments is larger, as a `bind` field, which later attribution of a postponed start takes as authoritative.

Reservations are written in decoder order; ties break by a fixed rule at both sites—the routing queue is keyed by (arrival time, node id) and updates only on a strictly earlier arrival; dispatching scans vehicles by increasing id and keeps the first strictly better one (Section 4.5). A chromosome therefore maps to a unique timetable, so the seed-paired comparisons of Section 5 are deterministic.

Proposition 2 (Every chromosome is decodable). *For any machine assignment and any operation sequence, Algorithm 2 terminates and returns a conflict-free, deadlock-free timetable of finite makespan. The search therefore need not invoke a repair operator.*

Proof. By the encoding, the operation sequence is a topological order of each job's precedence constraints: the x-th appearance of a job stands for its x-th operation, so ready_j is already known when it is read. Ample buffers (A4) mean that an operation is never blocked for lack of storage, so once the job has arrived, (10) always supplies a finite start time. Each Route call returns in finite time by the argument after Algorithm 1, and occupies only a still-free slot on the reservation table, so (8) holds by construction. Deadlock-freedom is likewise by construction: the corridor is the only contended resource (A6); waits occur only at ample berths and do not occupy corridors; every occupancy written to the table ends at a finite time, so there is no mutually waiting fleet—in the worst case a vehicle waits until every existing occupancy has ended and then proceeds. The sweep executes $N + n\delta_{\text{ret}}$ times, and each time schedules a finite number of events at finite times. □

The outer search is a conventional genetic algorithm, so the gains of Section 5 can be attributed to the mechanisms above rather than to operator tuning. The initial population is random (uniform arm from Ω_{ji}, shuffled sequence), plus two seeds: every operation on its fastest arm, and a load-balance greedy. Selection is tournament with a small elite; the sequence uses precedence-preserving order crossover, the assignment uniform crossover; mutation swaps two sequence positions or moves an operation to another optional arm. Fitness is the decoded makespan; hyperparameters are in Section 5.1.3.

4.4 Reservation-table-aware dispatching

A vehicle may be chosen either by an unimpeded-shortest-path estimate or by a trial query on the reservation table. The open-loop rule scores each vehicle by

$$\text{score}(k) = \max\big(\text{avail}_k + t^*(\text{loc}_k, \text{pickup}),\ \text{ready}_j\big) + t^*(\text{pickup}, \text{destination}), \tag{12}$$

which does not consult the reservation table and may therefore select a vehicle that is geometrically near but whose corridor is already occupied. The formula is the same as the idle-first dispatching of [10], and is a typical single-attribute rule in surveys of multi-AGV dispatching [12].

The closed-loop default is instead *reservation-table-aware dispatching*. For each candidate vehicle, the empty leg is planned and temporarily written to the reservation table, the loaded leg is then planned against that state, the earliest feasible delivery time is recorded, and the table is rolled back to its state before the trial query. The empty leg must be written temporarily; otherwise the loaded leg cannot see its occupancy, the two legs may be placed in the same corridor–time slot, and the trial query would be optimistically early. The vehicle with the earliest feasible delivery is selected, not the vehicle that (12) estimates as best.

Vehicles have identical capability (assumption A5) and differ only in current location and corridor occupancy, so contention on the reservation table determines the choice.

The trial query raises routing calls per transport from 2 to at most $2N_A$, about 12.2 times an unimpeded-time selection. Without the speed-ups of Section 4.5, this overhead offsets the makespan gain under an equal wall-clock budget; Section 5 therefore compares the two rules only under the same wall-clock limit.

4.4.1 Three choices for the inter-layer interface. With the two levels fixed, the remaining choice is what the lower level returns. This paper implements three interfaces and measures each under an equal wall-clock budget; only the first reduces makespan (Section 5).

Return realized durations only. The lower level returns traversal times after conflict-free routing—the evaluation path, and the quantity that a trial query reads. Every candidate is ranked by the makespan after yielding wait; schedule-then-route methods see that duration only after the assignment is fixed.

Return a surcharge on corridor–time slots. A shadow price $\pi(e, b)$ is attached to each occupied corridor–time slot; path planning then minimizes "arrival time $+\theta\cdot$cost" on the Pareto front ($\theta \geq 0$ is the only knob). Two estimates are implemented: perturbing one slot and re-solving, and a low-cost approximation refreshed each generation. Contention wait already enters arrival time and, through (10), makespan, so the surcharge adds no new information. It also adds cost: by Lemma 1, price is a second criterion, so single-label Dijkstra must become a multi-label Pareto search whether or not a surcharge is finally applied. Under an equal wall-clock budget makespan consistently worsens, and Section 5 attributes the loss mainly to this search overhead.

Return the yield record. A yield triple (corridor, time slot, preceding transport) generates the local-search neighbourhood of Section 4.6 without double-counting the delay. Under an equal wall-clock budget the neighbourhood is well targeted, but its hit rate does not offset the extra evaluations.

4.5 Low overhead: admissible lower-bound pruning and winner-path reuse

At equal generation count the trial query beats rule-based dispatching by 2.90%, but one evaluation is 12.2 times more expensive, so under equal wall-clock time the gain is offset and the net effect is almost zero. The two optimizations below lower the evaluation cost without changing any selection.

Admissible lower-bound pruning. Conflict-free routing can only become later because of yielding; it can never be faster than the unimpeded shortest path. The open-loop score (12) is therefore an *admissible lower bound* on the realized delivery time:

$$\mathrm{lb}(k) = \max\big(\mathrm{avail}_k + t^*(\mathrm{loc}_k, \mathrm{pickup}),\ \mathrm{ready}_j\big) + t^*(\mathrm{pickup}, \mathrm{destination}) \ \leq\ \mathrm{est}(k), \tag{13}$$

where $\mathrm{est}(k)$ is the earliest feasible delivery time obtained by a complete two-leg trial query for vehicle k. Hence if a vehicle's lower bound is already no earlier than the current best trial-query delivery, routing need not be called for it.

Winner-path reuse. After the trial query the reservation table is rolled back as a whole, the winner's two legs included; the decoder then recomputes them, wasting two routing calls. What Route$(\cdot, \mathit{commit} = \mathrm{true})$ does is merely write the legs into the reservation table in order, so the paths already computed for the winner during the trial query can be cached and committed directly, without replanning.

Proposition 3 (The two speed-ups do not change the selected vehicle or the timetable). *In the implementation, vehicles are queried in increasing id order, and the selected vehicle is replaced only when some vehicle's feasible delivery is strictly earlier than the current best. Under this rule, after the pruning of* (13) *and winner-path reuse are added, the selected vehicle, the occupancies written to the reservation table, and the final timetable are the same as when a complete two-leg trial query is run for every vehicle.*

Table 3. Critical-chain labels. Only `corridor` names a concrete corridor, time slot, and occupying transport.

Label	Criterion	Remedy
`operation`	a real operation is occupying its robotic arm	–
`machine`	`bind` = `machine`, blocked by the preceding operation on that arm	resequence
`upstream`	the vehicle arrived first, the material is not yet ready	recurse to $O_{j,i-1}$
`vehicle`	the material is ready, the vehicle is late, and yielding is a small share of the delay	change vehicle
`corridor`	a yield occurs on a specified corridor at a specified time slot	reassignment or staggering

Proof. Consider pruning first. When vehicle k is reached, let the current best trial-query delivery be est*. If $\mathrm{lb}(k) \geq \mathrm{est}^*$, then (13) gives $\mathrm{est}(k) \geq \mathrm{lb}(k) \geq \mathrm{est}^*$, so k's feasible delivery cannot be strictly earlier and would not be selected even if every vehicle received a complete trial query; not calling routing for it leaves the selected vehicle unchanged. The reservation table is saved before each trial query and restored afterwards; vehicles for which routing is not called do not rewrite the table and therefore do not affect later queries. Consider reuse next: the two legs of the selected vehicle were computed against the same reservation-table state; after restore, the table is the same state the decoder later writes into. Route$(\cdot, \mathit{commit} = \mathrm{true})$ only writes the given legs into the table, so writing the cached paths and replanning then writing produce identical occupancy intervals. □

Together the two optimizations give a 2.33-fold speed-up, cut routing calls by 47.4%, and reduce the cost multiple of a single evaluation from 12.2 to 5.23. These three numbers are the *joint* effect of the two: the experiment measures them as one on/off switch (Table 8), and this paper does not attribute them separately to pruning or reuse.

The effect of pruning varies with fleet size. The tighter the bound and the earlier a good trial-query delivery is obtained, the more vehicles need not call routing; the larger the fleet, the more candidates, and the total trial-query cost grows approximately linearly with N_A. Section 5 reports the measurement: the pruning effect weakens as the fleet grows, but still cuts 42.2% of the routing calls on the largest fleet tested. The two effects together mean that the gain of the mechanism may vanish after the fleet reaches some size; Section 5.4 explains why the present number of seeds is not sufficient to identify that size.

4.6 Yield-record-guided local search

The third interface of Section 4.4.1 is written as local-search operators on a typed critical chain (Table 3). Backtracking from the event that attains the makespan includes transport legs as well as operations, and labels each link as `operation`, `machine`, `upstream`, `vehicle`, or `corridor`. A yield on a loaded leg is always `corridor`; on an empty leg, a yielding share above 0.5 distinguishes `corridor` from `vehicle`. The same labels are used to read the case study of Section 5.7.

Two operators use the yield record. Reassignment scores each alternative arm $m' \in \Omega_{ji} \setminus \{m\}$ of at most $L = 5$ real operations on the chain by

$$\mathrm{score}(m') = t^*\big(\mathrm{pos}_{j,i-1}, \mathrm{loc}(m')\big) + t^P_{jim'}, \tag{14}$$

unimpeded travel plus processing time; the lowest-scoring arm produces a neighbour that changes only that assignment, which is the trade-off of this paper (a nearer arm may be slower). Staggering takes the longest `corridor` link and either advances the blocked operation or postpones the occupier; each round moves one operation, so the result remains a legal permutation and, by Proposition 2, remains decodable. The neighbourhood has at most $L + 2 = 7$ candidates. Each selected individual is given at most three first-improvement rounds, and the chain is recomputed after every acceptance. Under an equal wall-clock budget this interface does not yield a stable makespan gain (Section 5).

4.7 Complexity and cost

One fitness evaluation processes $N + n\delta_{\text{ret}}$ events. Under closed-loop dispatching each transport uses at most $2N_A$ routing calls, so each individual costs $O\big((N+n)\,N_A\,|E|\,(W^2 + \log|V|)\big)$; open-loop selection by unimpeded time no longer carries the factor N_A. The lower-bound pruning of Section 4.5 does not change this worst-case bound, but by Proposition 3 it reduces the actual number of calls by 47.4% without changing the output, i.e. it replaces N_A by a smaller effective multiple. Local search adds at most $3(L+2)$ further evaluations for each selected individual.

Within the framework, the cost of a single evaluation can differ by about two orders of magnitude. On the ladder batch the measurements are: open-loop 0.38 ms, closed-loop rule-based dispatching 2.70 ms, full closed-loop 14.2 ms, about 37 times the open loop. Pricing is not in that batch; it comes from an earlier pricing scan; within that batch, turning pricing on raises a single evaluation from 15.05 ms to 72.9 ms, another factor of about 4.8. The absolute milliseconds of the two batches must not be divided into each other; the product of the two multiples already shows the span: the most and least expensive configurations differ by about two orders of magnitude in the cost of a single evaluation. Equal generation counts are therefore not an appropriate comparison, and Section 5 aligns wall-clock time instead.

This framework does not prove convergence to a fixed point. The lower level plans sequentially, not jointly to optimality; the upper level is a randomized search on a non-convex space; what crosses the interface is a duration or a yield record, not a Lyapunov function. Proposition 2, Lemma 1, and Proposition 3 give a narrower guarantee that is enough for a heuristic: every candidate is feasible, its evaluation matches the conflict model of Section 3, and the two speed-ups do not change any selection; together with the deterministic decode of Section 4.3, the process is reproducible. Solution quality is given by the experimental protocol of the next section.

5 Experiments

This section measures the two paths separately under an equal wall-clock budget, then asks on which instances they hold and where the thicker interfaces fail.

5.1 Experimental setup

5.1.1 Controlled instance families. FJSP_PT benchmarks write travel times as a matrix and supply no corridor network. Of the 7 layouts used by Homayouni and Fontes [7], 7 matrices are entirely asymmetric and 2 of those also violate the triangle inequality, so they cannot be inverted into the undirected corridors used here without producing a different instance. This paper therefore generates controlled instances on an explicit road network, so that the features of Section 3.7 can be varied one at a time. Section 5.8 removes the contention constraint and compares on public instances; that batch calibrates the upper-level search, not the framework.

All instances share 12 jobs, 36 real operations, 8 robotic arms, $H = 0.3$, and $\bar{T}_t/\bar{T}_p$ scaled to 4.0, so that vehicle contention is the main constraint. H is standardized per operation so that changing F in Family C does not carry H with it. On this core the instances vary along three axes (Table 4).

Family A: layout. A dumbbell joins v_0 to the arm area by one or a few parallel lanes—the spine layout of [18]—so adding a lane changes capacity only, not distance; a tandem loop [17] is not used. `funnel`, `high`, and `mid` are dumbbells (node/corridor 13/12 to 15/16); `funnel` versus `high` differs by one station lane and halves the min-cut—the cleanest contrast. `low` and `scatter` are 4×4 grids (16/24) that differ only in arm placement, and supply the low-contention end a dumbbell cannot reach. The farthest unimpeded trip is 10.0 on every dumbbell and 18.0 on both grids, so only the

Table 4. Controlled instances. Shared core: 12 jobs, 36 operations, 8 arms, $H = 0.3$, $\bar{T}_t/\bar{T}_p \approx 4.0$. Contention intensity is yielding wait as a share of travel time on the initial random population.

Instance	Varied factor	Nodes	Corridors	N_A	F	Contention	Composite LB (time units)
A funnel	layout	13	12	12	0.61	31.9%	24.0
A high	layout	14	14	12	0.61	30.9%	17.0
A low	layout	16	24	12	0.61	11.8%	36.0
A mid	layout	15	16	12	0.61	10.8%	17.0
A scatter	layout	16	24	12	0.61	6.6%	36.0
B NA/NM 0.5	N_A/N_M	13	12	4	0.61	15.8%	24.0
B NA/NM 1.0	N_A/N_M	13	12	8	0.61	30.2%	24.0
B NA/NM 2.0	N_A/N_M	13	12	16	0.61	32.4%	24.0
C F=0.3	flexibility	13	12	12	0.30	27.7%	24.0
C F=1.0	flexibility	13	12	12	1.00	33.5%	24.0

three dumbbells can be attributed to capacity. Contention intensity runs from 31.9% to 6.6% and is not monotone (`low` is above `mid`); the paper-wide maximum 33.5% is the $F = 1.0$ instance of Family C.

Family B: vehicle-to-arm ratio N_A/N_M. On `funnel` take $N_A \in \{4, 8, 12, 16\}$, i.e. ratios 0.5, 1.0, 1.5, 2.0, of which 1.5 is the shared base point (listed as `A funnel`). Trial-query cost grows with the fleet and pruning weakens with it (Section 5.6).

Family C: flexibility F. On the same layout take $F \in \{0.3,\ 0.6,\ 1.0\}$, of which 0.6 is the base point (measured 0.61).

The three families comprise 10 instances: `A funnel` is generated once and occupies one row in Tables 4 and 7, so $5 + 3 + 2 = 10$; Section 5.4 must count that shared base when it reads Families B and C. Contention intensity is the mean share of yielding wait in travel time on the initial random population (20 individuals, seed 42), read off the instance, not an optimized solution. Relative to the 20 public instances of Section 5.8, the base $F = 0.6$ is about one and a half times the public median 0.38, $H = 0.3$ lies above the public maximum 0.258, and $\bar{T}_t/\bar{T}_p = 4.0$ is set above the public maximum 0.12 so that transport is worth modelling. Section 5.4 checks that H does not inflate the closed-loop gain, on a pre-speed-up batch of 16 instances, direction only.

5.1.2 Comparison protocol. Four rules govern every contrast. Every schedule is executed by the same conflict-free planner and checked by a validator independent of the decoder; the open loop's self-reported makespan is often unattainable (the gap between self-reported and realized values, hereafter the optimism margin, is especially large on the funnel layout), so what is compared is the realized makespan. A single evaluation can differ by about two orders of magnitude, so equal generation count would give the more expensive method more compute: the main batch shares a 90-second budget, a generation cap of 2000 (never fires), and early stopping after 400 generations without improvement. Early stopping fires on every open-loop run (a little over four hundred generations, one to three tenths of the budget); both closed-loop configurations run the full 90 seconds, which favours the baseline—the open loop is not cut off, and it completes as many evaluations as closed-loop rule-based dispatching, or more (Section 5.6). Companion batches have their own budgets, marked at first citation. Each instance is run with 10 seeds ($10 \times 10 = 100$ pairs) and a two-sided Wilcoxon signed-rank test on (instance, seed); 10 seeds resolve a few-percentage-point effect in the aggregate, but not inside a single instance (Section 5.4). Holm correction is by family: the five aggregate contrasts of Table 6 form one family; in Table 7 the 10 instances of one effect form a family each. Two exceptions are marked where they occur: vehicle-to-arm-ratio p-values in Section 5.4 are shown uncorrected because they do not reach that threshold; counts

Table 5. The 2 × 2 structure of the four-level ladder. Only a same-row or same-column contrast differs in one factor; a diagonal contrast cannot be decomposed.

	Rule-based dispatching	Reservation-table-aware dispatching
Open-loop planning (search ignores conflicts)	B0 [10]	B0+ [19]
Closed-loop planning (fitness from conflict-free routing)	B1	**B2** (this paper)

from the 16-instance batch are that batch's own uncorrected counts, direction only. Each aggregate contrast also reports a Hodges–Lehmann pseudomedian and a 95% distribution-free interval (same source as the rank test); the mean pairwise relative gain is convenient for adding across tables but carries no precision. Error bars in the figures are the SEM of that mean; tables do not report ±. Runs that fail the validator or fall below the composite lower bound are reported separately and are not folded into any instance mean.

5.1.3 Implementation and environment. The experiments run single-threaded on an Intel Xeon W-1270 @ 3.40 GHz, in pure Python (≥ 3.9.6), standard library only, no solver. Hyperparameters are defaults, untuned, and identical across configurations: population 60, crossover 0.8, mutation 0.2, elite 5, binary tournament; local search each generation on the top 10%, at most $L = 5$ critical operations per round and three rounds. Absolute runtimes cannot be compared with a compiled implementation: a faster code would let every configuration complete more evaluations inside the same budget, but would not change the exchange among configurations that share this implementation.

5.2 Four-level baseline ladder

The four levels form a 2 × 2 design (Table 5) along two factors: whether the search counts conflict-induced delay, and whether a vehicle is selected by unimpeded time or by a trial query. Ablations alone cannot be mapped onto published structures, while a literature comparison alone would credit the gain of conflict-free routing itself to our mechanisms. **B0** is open-loop planning plus realized execution, corresponding to [10] (idle-first, the same formula as (12)). **B0+** changes only vehicle selection on a frozen open-loop schedule, corresponding to the two-stage structure of [19]; the trial query serves as a greedy approximation of that second stage. **B1** is closed-loop planning with rule-based dispatching; **B2** is this paper. Only a same-row or same-column contrast differs in one factor; B0+ → B1 crosses both and cannot measure the closed loop.

B0 execution has two reproductions: *replay the plan* keeps the vehicles specified by the search; *on-site reassignment* re-selects them at execution time. Replay is stronger (63.00 versus 78.00 on the funnel instance) and matches [10]; the main report uses replay, and on-site reassignment is listed alongside.

5.3 Aggregate comparison on the ladder

Table 6 reports the four-level ladder on 10 instances × 10 seeds (100 pairs), equal wall-clock (Section 5.1.2); every run passed the independent check. Holm over the five contrasts is inert: the largest is $p = 0.0054$ before and after; the other four remain below 10^{-4}. The text writes the Hodges–Lehmann interval as a reduction (smaller to larger magnitude); the table writes the same interval with a minus sign.

Fixing the dispatching rule, the closed loop is significant under both rules: B0 → B1 is −18.86% ($p < 10^{-4}$; HL 18.28%, 95% CI [14.68%, 21.95%]), and B0+ → B2 is −17.75% ($p < 10^{-4}$). The former interval lies far from zero and its lower end is still a two-digit reduction. The open loop completes several times as many evaluations inside the same budget (Section 4.5) and is still worse: it optimizes a surrogate that excludes yielding wait.

Table 6. Four-level ladder under an equal wall-clock budget, and the four one-factor pairs. Negative is better. $n = 100$ pairs on (instance, seed); two-sided Wilcoxon signed-rank; Holm over the five contrasts ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$). HL is the Hodges–Lehmann pseudomedian and its 95% interval (same source as the test; the mean is for adding across tables, Section 5.1.2). A bold header marks this paper's configuration, not a per-instance winner (Table 7).

		$C_{\max}$ (time units)			
Instance	Contention	B0	B0^{+}	B1	**B2**
A funnel	31.9%	58.10	53.50	44.70	42.40
A high	30.9%	56.00	51.40	39.90	37.70
A low	11.8%	64.50	64.60	57.80	57.10
A mid	10.8%	46.40	42.70	39.40	37.40
A scatter	6.6%	62.30	62.30	57.20	57.10
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20
C F=0.3	27.7%	74.30	69.50	51.60	47.30
C F=1.0	33.5%	52.10	47.50	39.30	36.30
Pooled mean	—	61.20	58.27	48.86	47.44

Single-factor paired contrast	$\Delta C_{\max}$	HL pseudomedian and 95% CI (%)	W/L/T	p_{Holm}
Closed-loop value (rule-based)	$-18.86\%^{***}$	-18.28 [-21.95, -14.68]	96/2/2	$< 10^{-4}$
Closed-loop value (reservation-table-aware)	$-17.75\%^{***}$	-18.00 [-20.55, -15.22]	93/5/2	$< 10^{-4}$
Dispatching value (open-loop)	$-4.59\%^{***}$	-4.45 [-6.13, -2.88]	68/19/13	$< 10^{-4}$
Dispatching value (closed-loop)	$-2.45\%^{**}$	-3.03 [-5.19, -0.94]	60/34/6	0.0054
End-to-end: literature → this paper	$-21.35\%^{***}$	-21.20 [-24.46, -17.99]	94/5/1	$< 10^{-4}$

Fixing the loop, reservation-table-aware dispatching inside the closed loop is -2.45% ($p = 0.0054$; HL 3.03%, 95% CI [0.94%, 5.19%]). The two levels share the same search and budget and differ only in the trial query. This is a pooled result: the interval does not cross zero, but its lower end is less than one percentage point from zero; after Holm correction no instance is significant individually (2 before correction), and at ratios 1.0 and 2.0 B2 is slower than B1 (Table 7). Thus -2.45% is a pooled mean on this family, not a per-instance point estimate (Section 5.4). Under the open loop the same rule is -4.59% ($p < 10^{-4}$), significant but far smaller than 18.86%, and corresponds to the two-stage segment of [19] relative to [10].

End-to-end, B0 → B2 is -21.35% ($p < 10^{-4}$; 94 wins, 5 losses, 1 tie). If the two factors were independent, the compound of 18.86% and 4.59% would be 22.6%; the measurement is slightly below that—no positive interaction, near-additive. Open-loop dispatching cannot replace the closed loop: a two-stage structure of the [19] kind reaches 4.59%; the closed-loop reduction requires yielding wait in fitness. B2 is best end-to-end because it stacks the two, not because they amplify each other. The gap of schedule-then-resolve is not the second-stage 4.59%, but that the first stage never sees yielding wait.

5.4 Three-factor scans and per-instance resolution

Table 7 and Figure 3 split the 10 instances by layout, N_A/N_M, and F. The closed loop can be read as a trend by family; dispatching cannot be resolved on a single instance. Per-instance tests are Holm-corrected by family: the family is the 10 instances of one effect, not the twenty pooled. After correction the closed loop remains significant on 9 instances

Table 7. Per-instance ladder and the two effects, blocked by family ($n = 10$ seed-pairs). Stars are Holm-corrected over the 10 instances of one effect ($^{*}p < 0.05$, $^{**}p < 0.01$); † is significant only before correction and is not read as a conclusion. Negative is better. Gain columns use the same pairwise-mean estimator as Table 6. `A funnel` is the shared base point of Families B and C and occupies one row.

		$C_{\max}$ (time units)				Pairwise relative gain	
Instance	Contention	B0	B0^{+}	B1	**B2**	B0→B1	B1→B2
A funnel	31.9%	58.10	53.50	44.70	42.40	−21.3%*	−4.6%
A high	30.9%	56.00	51.40	39.90	37.70	−26.6%*	−4.8%
A low	11.8%	64.50	64.60	57.80	57.10	−10.3%*	−1.1%
A mid	10.8%	46.40	42.70	39.40	37.40	−14.4%*	−4.8%
A scatter	6.6%	62.30	62.30	57.20	57.10	−8.1%*	+0.1%
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10	−2.0%	−4.7%
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80	−15.6%*	+5.0%
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20	−37.7%*	+5.7%
C F=0.3	27.7%	74.30	69.50	51.60	47.30	−29.7%*	−8.2%†
C F=1.0	33.5%	52.10	47.50	39.30	36.30	−22.9%*	−7.2%†

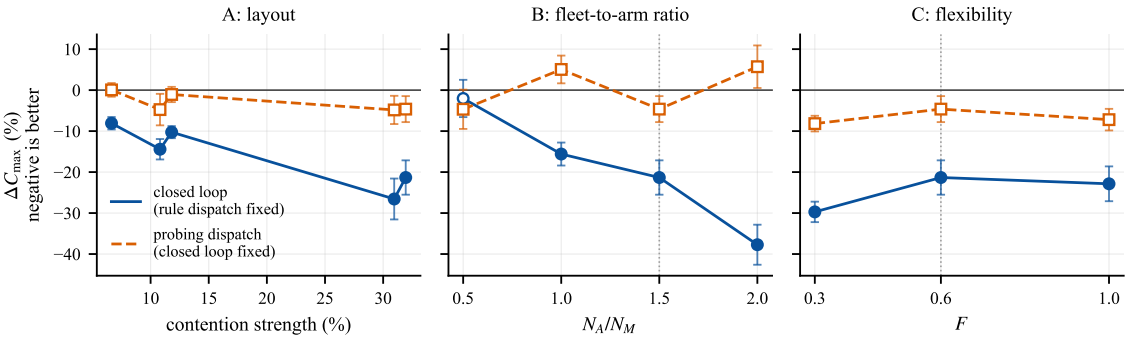


Fig. 3. Closed-loop and dispatching gains on the three families. Left: layout versus contention intensity; middle: N_A/N_M; right: flexibility. Hollow vertical lines mark the shared base point `A funnel`. A filled marker remains significant after Holm correction over the 10 instances of that effect; a hollow marker does not. Error bars are the SEM of pairwise relative gains over 10 seeds. Read whether each curve can be distinguished from zero, not the shape of the dispatching curve.

(uncorrected 9; largest corrected site $p = 0.0401$); the only exception is the tightest fleet (B0 → B1 is -2.0%). Dispatching is significant on 2 instances before correction (the two ends of Family C, $p = 0.0105$ and $p = 0.0321$) and on 0 after (Holm $p = 0.1052$, $p = 0.2885$; Benjamini–Hochberg $p = 0.1052$, $p = 0.1603$, likewise not significant). A few percentage points is of the same order as the smallest effect that 10 seeds on one instance can resolve; only the 100 pooled pairs distinguish it from noise (pseudomedian 3.03%, 95% interval [0.94%, 5.19%], lower end less than one percentage point from zero). Per-instance dispatching results below are reported as directions only.

Layout. The closed loop is significant on all five instances and rises roughly with contention: -8.1% on `scatter` (6.6%), -14.4%/-10.3% on `mid`/`low`, -21.3%/-26.6% on `funnel`/`high`. Two inversions block a monotone ranking. `low` versus `mid` crosses the grid/dumbbell boundary of Section 5.1 (18.0 versus 10.0) and is not a pure contention contrast. `funnel` versus `high` differs only by one station lane—the designated cleanest contrast—and is the inversion that is a counter-example.

What remains is the span (-21.3% versus -8.1%, about two and a half times), not an ordering; contention intensity does not say how much wait can be bypassed by reassignment (limitation (1) of Section 6.2). Dispatching is not significant on any instance of this family; four of five signs match the closed loop.

Vehicle-to-arm ratio. This family does not determine an upper bound of applicability for dispatching. Dispatching at the four levels is −4.7% (0.5), +5.0% (1.0), -4.6% (1.5), +5.7% (2.0): the sign alternates, and none is significant even before correction ($p = 0.4121$, $p = 0.2011$, $p = 0.1386$, $p = 0.3313$). Without the base point the curve would show the monotone rise predicted by the cost law; when an effect is of the same order as per-instance noise, subset selection alone can produce a spurious trend. The cost-law upper bound of Section 5.6 therefore remains untested. The closed loop at 1.0 and 2.0 is -15.6% and -37.7%, both significant: fleet size mainly affects the closed-loop gain, not dispatching.

Flexibility. After Holm correction, dispatching is not significant in this family either. At $F = 0.3$, 0.6, and 1.0 the gains are -8.2%, -4.6%, and -7.2%, all of the same sign; the two ends are the largest among the 10 instances but are no longer significant after correction ($p = 0.1052$, $p = 0.2885$), so the family provides directional evidence only. The same sign at $F = 0.3$, where the room for reassignment is smallest, is compatible with dispatching acting mainly through vehicle selection and timing rather than through reassignment to another arm. The closed loop is -29.7% and -22.9%, still significant per instance after correction.

An independent batch, and heterogeneity is not an amplifier. A contended cross-check (16 instances × 10 seeds, two-stage surrogate baseline, per-instance wall-clock 30.2–72.9 s) gives an integration gain of 5.52%, significant on 11 instances (uncorrected counts from that batch; the second exception of Section 5.1.2), from 0.28% to 9.28%. By H bin the same gain is 6.12% at $H = 0$, 5.62% at 0.15 (measured 0.139, near the public median 0.135), 5.95% at 0.3, and 4.38% at 0.5; the rank correlation is a weakly negative -0.21. H above the literature range did not inflate the closed-loop gain; $H = 0$ is the highest bin, consistent with Section 3.7. Dispatching on that batch is -0.74%, significant on 0 instances; the most likely reason is that the batch ran before the speed-ups of Section 4.5 (Section 5.6).

5.5 Ablations and controlled negative controls

Of the three interfaces in Section 4.4.1, only reservation-table-aware dispatching has a net gain under an equal wall-clock budget. On the 16-instance data of Section 5.4, turning off yield-record feedback is -1.72% (significant on 1 instance) and turning off staggering is -0.16% (significant on 2). Neither ablation worsens makespan, so the evaluations spent on these operators are better used by the population cycle. This earlier batch may be cited for these two ablations—their cost does not depend on the speed-ups—but not for dispatching. A θ scan on two $8 \times 4 \times 4$ instances (not Families A/B/C), collected before the speed-ups, is consistent with the observation of Section 4.4.1 that the surcharge adds no new information: under equal wall-clock every θ worsens makespan, flat over a ten-fold range; under equal evaluations the same comparison cannot be distinguished from zero. Priced routing completes about one fifth as many evaluations, so the measured loss comes mainly from the multi-label search overhead of Lemma 1, so θ is not a useful parameter.

The yield-record neighbourhood is small and directed (Section 4.6), yet fewer than one in fifty neighbours improve makespan; three repairs of the operators were not significant under an equal number of evaluations. Removing one corridor delay promotes another constraint, so the realized improvement is only a fraction of the wait removed (Figure 5)—the same dilution that left dispatching in the noise at 12.2 and significant after the multiple fell to 5.23. The default therefore keeps dispatching and does not use the yield record to generate the neighbourhood.

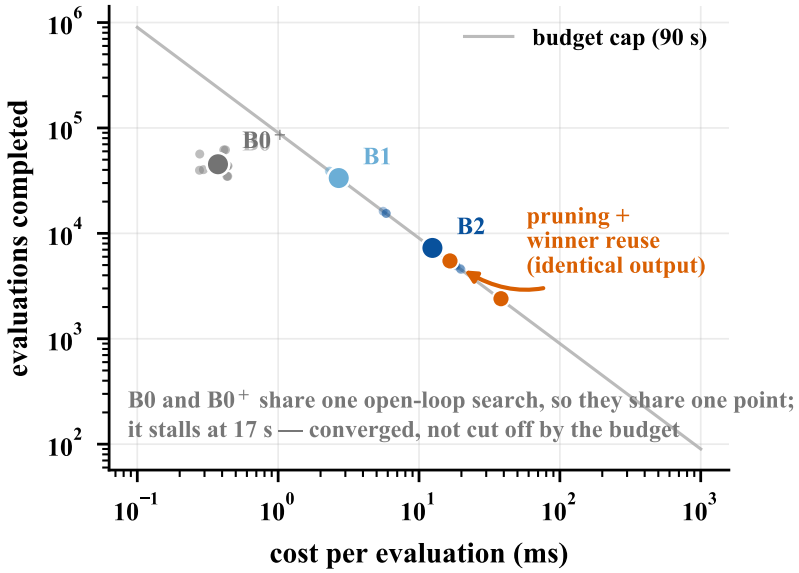


Fig. 4. Equal wall-clock time locks cost per evaluation and evaluations completed onto a hyperbola; the grey line is the shared 90-second cap. Closed-loop runs sit on the line; the open loop sits below it (early stop after ~17 s, Section 5.1.2). B0 and B0$^+$ share a point. Pale points are per-instance measurements on three instances; filled markers are their geometric mean, which preserves cost × count on a log axis. Orange markers are pruning and winner-path reuse off and on (callout at the on setting): Proposition 3 guarantees that the output does not change.

5.6 The cost law and the effect of the speed-ups

The same dispatching rule is indistinguishable from noise before the speed-ups and significantly positive after them, so the speed-ups themselves are measured here. Figure 4 locks cost per evaluation and evaluations completed onto a shared wall-clock hyperbola. The cheaper configuration completes more evaluations; the open loop uses less than two tenths of the wall-clock to complete the most. Before the speed-ups, reservation-table-aware dispatching is 2.90% ($p = 0.0269$) at equal generation count and 0.03% ($p = 0.8799$) at equal wall-clock—almost zero. Equal generation count asks whether the extra routing step improved that generation's ranking; equal wall-clock asks whether makespan fell in the same stretch of time. Both questions are well posed, but they must not be conflated.

Reservation-table-aware dispatching raises the routing calls per transport task from 2 to at most $2N_A$, so cost grows linearly with fleet size. The pruning bound is tighter when contention is weaker; the larger the fleet and the stronger the contention, the smaller the cut. The self-check of Section 4.5 gives this direction: of six instances the highest cut is on the least congested one (61.2%) and the lowest on the largest fleet (42.2%). The two trends *predict* that when the fleet is large enough and contention strong enough, trial-query cost will offset the gain. On the four N_A/N_M levels of Section 5.4 the sign alternates and none is significant, so the upper bound is not confirmed. The cost law itself (cost linear in N_A, pruning effect falling with N_A) is measured; the upper bound deduced from it is not.

The speed-ups and the dispatching rule were introduced one after the other, so a control that moves only the speed-ups is run: same instances, seeds, and budget. Proposition 3 guarantees that the two optimizations do not change the output. On each of six instances six random chromosomes give makespans and dispatch sequences *identical bit for bit* (36 groups, no exception); routing calls fall by 47.4%. Turning the optimizations off only makes one evaluation

Table 8. Pruning and winner-path reuse off versus on (bold header: the setting used here). Output is bit-identical, so the difference is saved compute. Same instances, seeds, and wall-clock; $n = 60$ pairs, two-sided Wilcoxon ($^{***}p < 0.001$). Ratio columns take the off setting as denominator, in opposite directions.

		$C_{\max}$ (time units)			Eval. count ratio	Speedup per eval.
Instance	Contention	Off	**On**	$\Delta C_{\max}$	(on/off)	(off/on)
A funnel	31.9%	44.80	42.10	−6.03%	2.21×	2.23×
A high	30.9%	41.60	37.70	−9.37%	2.40×	2.41×
A low	11.8%	60.50	57.00	−5.79%	2.79×	2.81×
B NA/NM 0.5	15.8%	66.70	65.10	−2.40%	2.11×	2.11×
B NA/NM 1.0	30.2%	51.90	48.90	−5.78%	2.31×	2.31×
B NA/NM 2.0	32.4%	44.50	42.20	−5.17%	1.98×	2.01×
Pooled (60 pairs)	—	—	—	−5.60%***		50/0/10, $< 10^{-4}$
HL pseudomedian and 95% CI: −4.72 [−6.34, −3.55] %						

slower, so the equal-wall-clock difference comes only from saved compute. Table 8: off→on reduces makespan by a further 5.60% (50 wins / 0 losses, $p < 10^{-4}$); evaluations inside the same budget are 2.23 times, and a single evaluation is sped up by 2.33. The same rule is -0.74% on the pre-speed-up matrix batch (significant on 0 of the 16 instances) and significantly positive on the post-speed-up ladder; the step in between did not change any selection.

5.7 Case study: Gantt chart and critical chain on the funnel instance

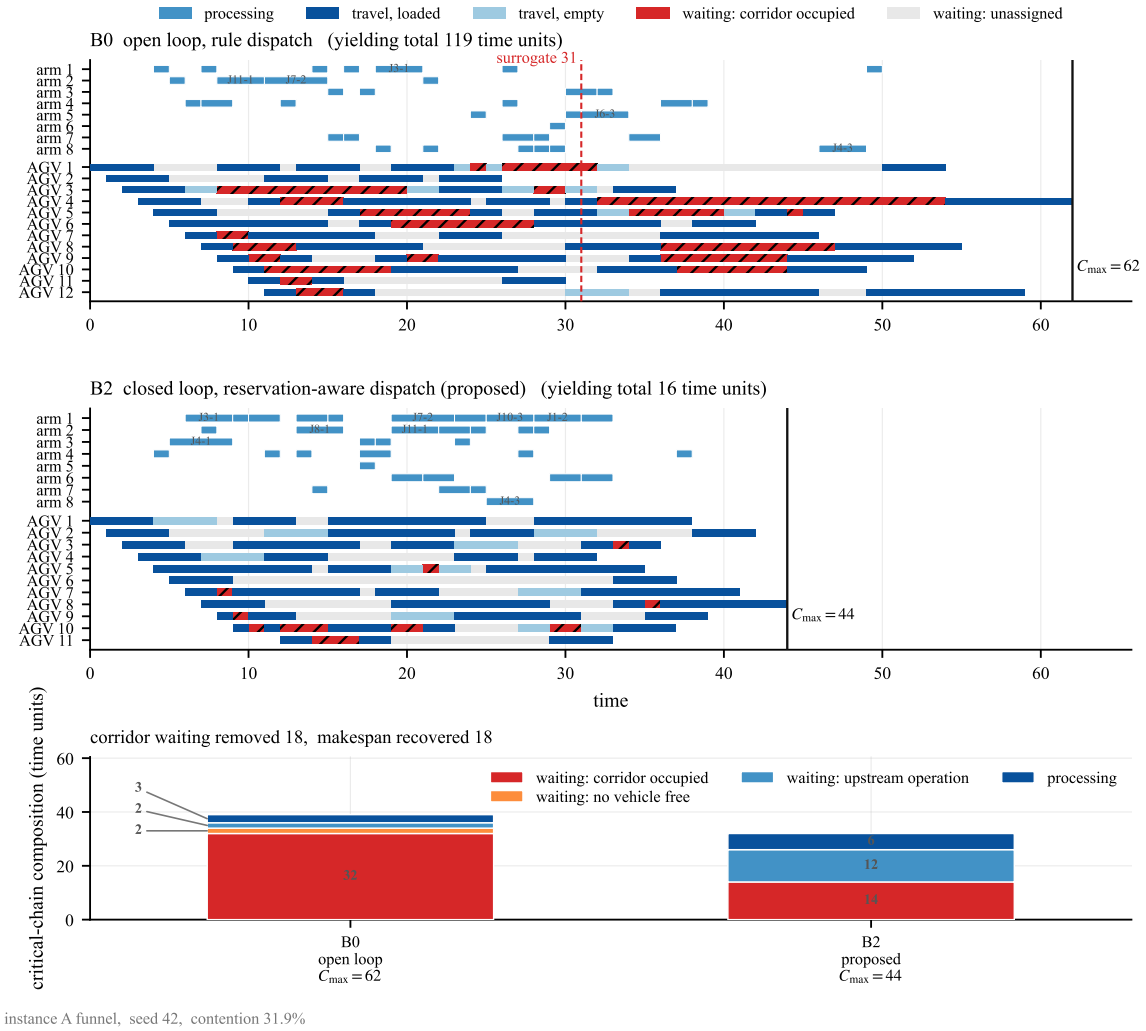
Figure 5 places B0 and B2 on one funnel instance: arms and vehicles share a time axis, and the critical chain is coloured by Table 3. In the open-loop solution, yielding wait appears near the station and concentrates on a few vehicles. The closed-loop solution cuts those waits and assigns several operations to an arm that is not the fastest—assignments of the (14) kind—and, as a consequence rather than a third mechanism, balances vehicle idle and busy time more evenly. On B0's chain `corridor` occupies a substantial share; on B2 those links largely disappear, and `machine` and `upstream` come to the top. That is the dilution of Section 5.5: remove one corridor delay, and another constraint becomes tight.

5.8 Degenerate comparison on public benchmarks

The preceding sections compared configurations of the same codebase; they cannot calibrate the absolute level of the implementation. After (8) is removed the model degenerates to standard FJSP_PT, the *same mathematical problem* as the public benchmarks; $\delta_{\text{ret}} = 0$ aligns the makespan definition (Table 2). Neither mechanism enters; what is tested is the *upper-level search*.

The 20 instances compiled by Homayouni and Fontes [7] (originating in Fattahi et al. [2]) are taken with two vehicles as in the literature [11] and no return of finished goods. Travel times are used as published: 2 of the matrices are not a shortest-path closure, and a synthesis would change the instance. 16 instances have a MILP-proven optimum [7]; the remaining 4 take the published best-known [11].

Table 9 is 20 instances × 10 seeds. All 200 runs pass the validator and the composite lower bound; on the 16 proven-optimum instances no run falls below that value. On 12/20 instances the best solution is identical bit for bit to the reference, including all 10 small instances; the mean gap is 2.68% (median 0.00%). The four largest instances have mean 9.96%, worst 12.93%; none of the 16 instances with at most 24 operations exceeds 5.45%.



instance A funnel, seed 42, contention 31.9%

Fig. 5. B0 and B2 on one funnel instance (existence reading, not a statistical claim). Top: arms and vehicles on a shared time axis; a gap inside one transport is yielding wait, a gap between tasks is idle time. Bottom: critical-chain composition by Table 3; corridor wait removed versus makespan recovered is the dilution of Section 5.5.

Raising the budget on the 10 MFJST instances (2.5 times the wall-clock, generation cap five times) cuts their mean gap from 5.37% to 4.53% (0.84 percentage points); none of the 300 runs hits the generation cap—a capability boundary, not a short budget (item (3) of Section 6.2). On the 16 proven-optimum instances the composite lower bound lies a median 10.3% below the optimum and 10.8% below this paper's best; the search shortfall is a median 0.0 percentage points and at most 4.1, so almost all of the gap is slack in the bound. The self-generated instances cannot inherit this proportion: their bound contains no contention, so 42% *must not* be read as a search gap (limitation (3) of Section 6.2).

Table 9. Degenerate comparison on public benchmarks (exclusivity removed). OPT: MILP-proven optimum; BK: published best-known; gap from the best of 10 seeds. An asterisk marks a bitwise-identical best solution.

			C_{max} (time units)			
Instance	Ops	Ref.	Value	Best	Mean	Gap
SFJST01	4	OPT	70	70*	70.0	0.00%
SFJST02	4	OPT	111	111*	111.0	0.00%
SFJST03	6	OPT	223	223*	223.0	0.00%
SFJST04	6	OPT	359	359*	359.0	0.00%
SFJST05	6	OPT	123	123*	123.0	0.00%
SFJST06	9	OPT	324	324*	324.0	0.00%
SFJST07	9	OPT	409	409*	409.0	0.00%
SFJST08	9	OPT	269	269*	269.5	0.00%
SFJST09	9	OPT	220	220*	222.0	0.00%
SFJST10	12	OPT	531	531*	531.0	0.00%
MFJST01	15	OPT	485	485*	485.0	0.00%
MFJST02	15	OPT	463	476	483.4	2.81%
MFJST03	18	OPT	482	482*	504.7	0.00%
MFJST04	21	OPT	576	586	623.0	1.74%
MFJST05	21	OPT	532	561	592.8	5.45%
MFJST06	24	OPT	652	677	728.3	3.83%
MFJST07	32	BK	898	965	1023.3	7.46%
MFJST08	36	BK	900	970	1027.0	7.78%
MFJST09	44	BK	1098	1226	1279.2	11.66%
MFJST10	48	BK	1230	1389	1473.0	12.93%

6 Conclusion

Flexible assignment and conflict-free AGV routing must be decided jointly: travel time is produced by routing, and when the corridor to a fast arm is congested, a slower, better-connected arm may be the better assignment. Existing methods schedule on a constant matrix or freeze the assignment first, so yielding wait does not enter the evaluation of candidates. Our bilevel framework connects the two levels by an evaluation path and a decision path and measures them separately under the same wall-clock budget; because a conflict-free evaluation is more than an order of magnitude more expensive than a matrix lookup, it also includes output-preserving speed-ups.

6.1 Findings and contributions

The four contributions of Section 1 are restated with their supporting numbers.

(1) **Instances.** On top of the problem setting of [21], instance families with controllable contention intensity characterize how high contention must be before counting yielding wait in the schedule yields a gain (Section 5.4).

(2) **Evaluation.** Routing-in-the-loop evaluation reduces makespan by 18.86% under rule-based dispatching ($p < 10^{-4}$) and by 17.75% under reservation-table-aware dispatching ($p < 10^{-4}$). Proposition 2 and Lemma 1 guarantee that every chromosome is decodable and that the evaluation matches the conflict model of Section 3.

(3) **Dispatching.** Within the closed loop, trial-querying both legs reduces makespan by a further 2.45% ($p = 0.0054$). Admissible lower-bound pruning and winner-path reuse change neither the selected vehicle nor the timetable (Proposition 3), cut router calls by 47.4%, and lower the cost multiple from 12.2 to 5.23.

(4) **Evidence.** The two paths differ in magnitude, are near-additive, and are not substitutes. The closed loop remains significant after Holm correction on 9 of the 10 instances; dispatching is significant only when pooled (95% interval [0.94%, 5.19%]) and on no instance individually. The end-to-end 21.35% is close to the prediction under independent composition, 22.6% (no positive interaction detected); open-loop dispatching (4.59%) cannot replace the closed loop. A surcharge on occupied corridor–time slots, and reassignment or staggering from the yield record, do not further reduce makespan under the same budget.

6.2 Limitations and future work

(1) Instance families. The main experiment fixes size and varies only structure (Section 3.7), so we draw no conclusion about how the mechanisms vary with size. H is fixed at 0.3; "the gain does not rise with H" comes from the smaller independent batch of Section 5.4 and holds only at that size. The layout family is spliced from two topologies: the three dumbbells change only capacity, while the two grids change topology and distance together (10.0 versus 18.0, Section 5.1). Contention intensity is a one-dimensional measure that does not separate the wait reassignment can bypass, so only the differences among the three dumbbells can be attributed to capacity alone. Self-generated instances are used because no usable public benchmark exists: common FJSP_PT benchmarks publish constant matrices, 7/7 of the examined matrices are asymmetric and 2 are not a shortest-path closure (Section 5.1), and works with an explicit road network use different reservation units (Section 3.5). Future work should remeasure on a larger corridor network and treat "whether congestion can be bypassed by reassignment" and "whether the critical chain has a single dominant bottleneck" as separate factors; φ failed to separate the corresponding layout classes.

(2) Dispatching is positive on average, but its per-instance stability is not yet determined. 10 seeds suffice for the closed loop (9/10 after Holm). The effect of reservation-table-aware dispatching is a few percentage points, of the same order as the seed-to-seed dispersion: 2/10 instances are significant before correction and 0/10 after, so the gain is confirmed as positive only on the 100 pooled pairs (95% interval [0.94%, 5.19%]). About an order of magnitude more seeds, and denser N_A/N_M levels, would be needed to determine the class of instances on which it is stable and to test the cost-law upper bound of Section 5.6.

(3) The distance of solutions from the optimum under corridor contention is unknown. The configurations share one code base, so the strength of the upper-level search does not affect the differences among levels. Section 5.8 is bitwise identical to the literature on 12/20 instances, and all 10 small instances reach a proven optimum. The four largest average 9.96%, and 2.5 times the compute improves this by only 0.84 percentage points, so the gap reflects a capability limit rather than an insufficient budget. On the 16 proven-optimum instances the search solution is at most 4.1 percentage points from the optimum, and the bound gap comes almost entirely from slack in the bound. On the self-generated instances the bound contains no contention, so its median gap 42% cannot be read as a search gap; a secondary effect of a few percentage points such as 2.45% may be of the same order as the unknown distance to the optimum. Exact solutions by logic-based Benders decomposition [21] on small contended instances would separate bound slack from search shortfall.

(4) The surcharge and yield-record-guided local search must be remeasured if their own cost is reduced. Both were assessed in the same code base under the same wall-clock budget, so what was measured is the trade-off between cost per evaluation and number of evaluations. Speeding up all configurations by the same factor leaves that trade-off and the sign unchanged; only reducing the cost of one interface separately could change the sign, as the dispatching speed-ups of Section 4.5 illustrate. Re-solving only the perturbed suffix of a chromosome would allow that local search to be remeasured under the same wall-clock budget.

(5) Failures and online arrivals are out of scope. Assumption A7 excludes robotic-arm and vehicle failures and dynamic arrivals from the model. The reported results hold for a static horizon; with failures or online arrivals, the value of a mechanism also includes how fast it repairs a schedule [1, 6, 23].

References

[1] Stéphane Dauzère-Pérès, Junwen Ding, Liji Shen, and Karim Tamssaouet. 2024. The Flexible Job Shop Scheduling Problem: A Review. *European Journal of Operational Research* 314, 2 (2024), 409–432. doi:10.1016/j.ejor.2023.05.017

[2] Parviz Fattahi, Mohammad Saidi Mehrabad, and Fariborz Jolai. 2007. Mathematical Modeling and Heuristic Approaches to Flexible Job Shop Scheduling Problems. *Journal of Intelligent Manufacturing* 18, 3 (2007), 331–342. doi:10.1007/s10845-007-0026-8

[3] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. 2021. Planning and Control of Autonomous Mobile Robots for Intralogistics: Literature Review and Research Agenda. *European Journal of Operational Research* 294, 2 (2021), 405–426. doi:10.1016/j.ejor.2021.01.019

[4] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

[5] Jianqi Gao, Yanjie Li, Xinyi Li, Kejian Yan, Ke Lin, and Xinyu Wu. 2023. A Review of Graph-Based Multi-Agent Pathfinding Solvers: From Classical to Beyond Classical. *Knowledge-Based Systems* 283 (2023), 111121. doi:10.1016/j.knosys.2023.111121

[6] Yong Gui, Dunbing Tang, Haihua Zhu, Yi Zhang, and Zequn Zhang. 2023. Dynamic Scheduling for Flexible Job Shop Using a Deep Reinforcement Learning Approach. *Computers & Industrial Engineering* 180 (2023), 109255. doi:10.1016/j.cie.2023.109255

[7] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[8] Kazumi Kasaura, Mai Nishimura, and Ryo Yonetani. 2022. Prioritized Safe Interval Path Planning for Multi-Agent Pathfinding with Continuous Time on 2D Roadmaps. *IEEE Robotics and Automation Letters* 7, 4 (2022), 10494–10501. doi:10.1109/LRA.2022.3187265

[9] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[10] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[11] Che Han Lim and Seung Ki Moon. 2023. A Two-Phase Iterative Mathematical Programming-Based Heuristic for a Flexible Job Shop Scheduling Problem with Transportation. *Applied Sciences* 13, 8 (2023), 5215. doi:10.3390/app13085215

[12] Wenhao Lu, Shuai Guo, Tao Song, and Yuwen Li. 2022. Analysis of Multi-AGVs Management System and Key Issues: A Review. *Computer Modeling in Engineering & Sciences* 131, 3 (2022), 1197–1227. doi:10.32604/cmes.2022.019770

[13] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[14] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[15] Jacqueline Otala, Alden Minard, Golshan Madraki, and Seyedamirabbas Mousavian. 2021. Graph-based modeling in shop scheduling problems: Review and extensions. *Applied Sciences* 11, 11 (2021), 4741. doi:10.3390/app11114741

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Behnam Rahimikelarijani, Hamed Fazlollahtabar, and Sina Nayeri. 2020. Multi-Objective Multi-Load Tandem Autonomous Guided Vehicle for Robust Workload Balance and Material Handling Optimization. *SN Applied Sciences* 2 (2020), 1032. doi:10.1007/s42452-020-3002-y

[18] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[19] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[20] Karim Tamssaouet and Stéphane Dauzère-Pérès. 2023. A General Efficient Neighborhood Structure Framework for the Job-Shop and Flexible Job-Shop Scheduling Problems. *European Journal of Operational Research* 311, 2 (2023), 455–471. doi:10.1016/j.ejor.2023.05.018

[21] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[23] Hao Xiong, Shiqing Shi, Dongxu Ren, and Jialong Hu. 2022. Dynamic Scheduling for Flexible Job Shop with New Job Insertions by Deep Reinforcement Learning. *Applied Soft Computing* 127 (2022), 109360. doi:10.1016/j.asoc.2022.109360

[24] Fayong Zhang, Rui Li, and Wenyin Gong. 2024. Deep Reinforcement Learning-Based Memetic Algorithm for Energy-Aware Flexible Job Shop Scheduling with Multi-AGV. *Computers & Industrial Engineering* 189 (2024), 109917. doi:10.1016/j.cie.2024.109917

[25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.

Received XX XX 2026; revised XX XX 2026; accepted XX XX 2026